

COM 3240

REINFORCEMENT LEARNING

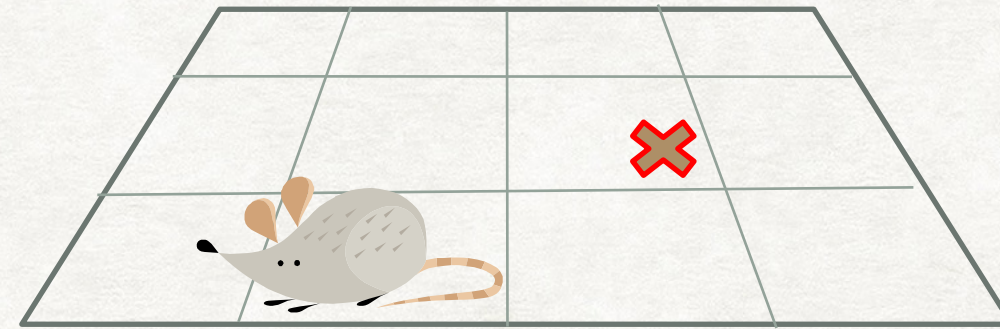


Create me a painting in the style of Dali with the title "waiting for the discounted expected returns"

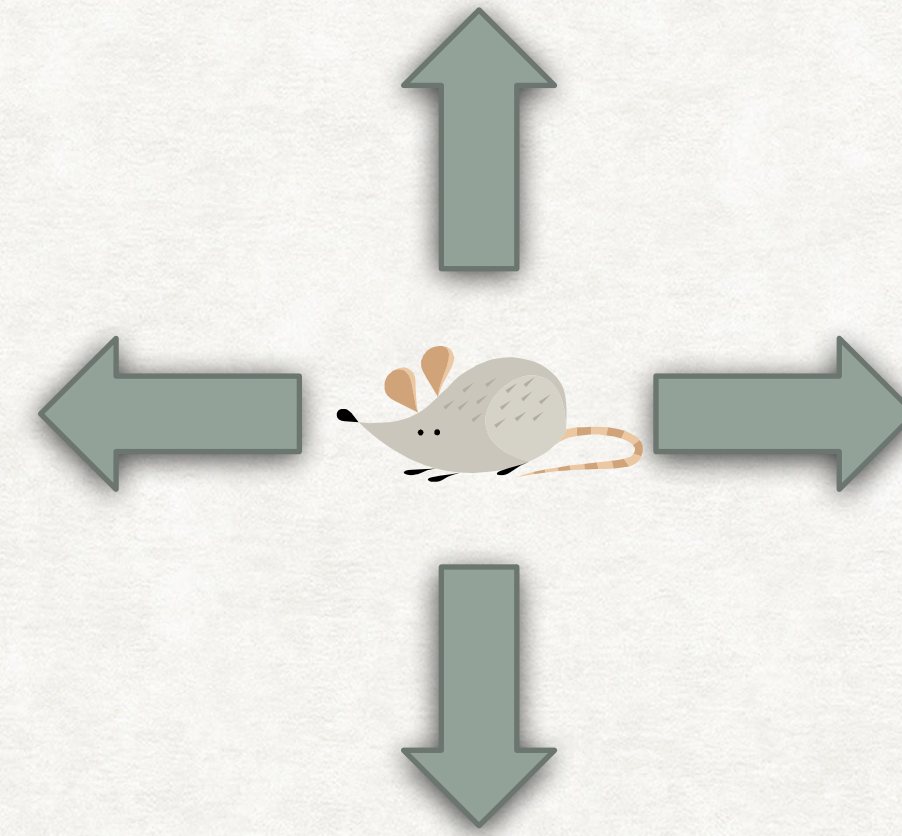
DALL·E

FUTURE REWARDS

MARCOVIAN PROPERTY



Squares = State (in this case)



Actions

The future state depends ONLY on the current state and not on the sequence of events that preceded it.

FUTURE REWARDS

EXPECTED RETURN

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

FUTURE REWARDS

EXPECTED RETURN

$$0 \leq \gamma < 1$$



$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

$$G_t = R_{t+1} + \gamma G_{t+1}$$

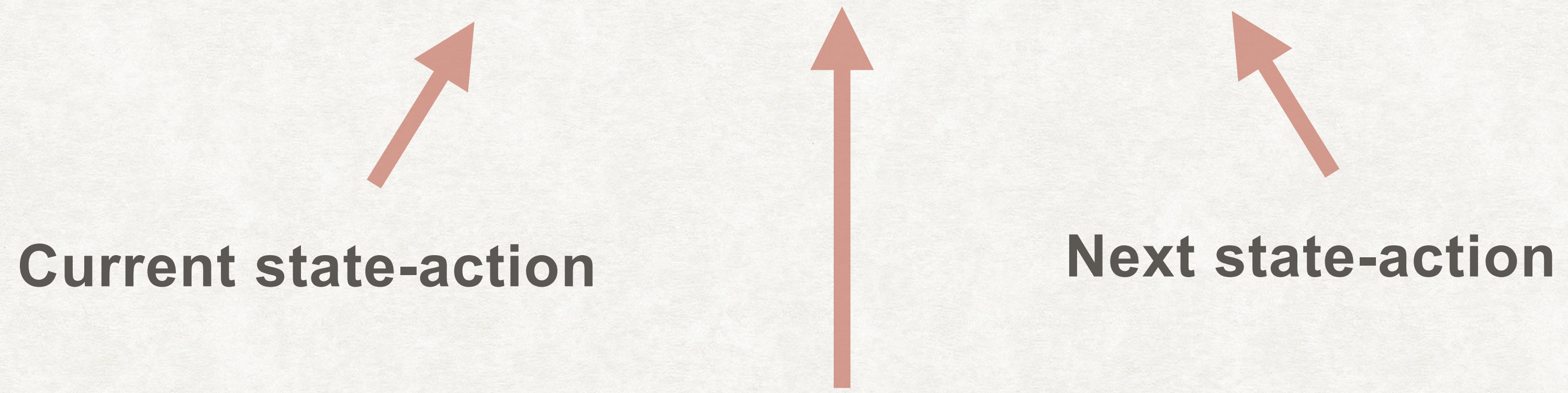
DALL·E

FUTURE REWARDS

OPTIMALITY & BELLMAN EQUATION FOR ACTION-VALUES

$$q^{\pi}(s, a) = \mathbb{E}_{\pi}[R_{t+1} + \gamma q^{\pi}(s', a') \mid S_t = s, A_t = a]$$

Simplify notation: $q^{\pi}(s, a) = \mathbb{E}_{\pi}[r + \gamma q^{\pi}(s', a') \mid s, a]$



Current state-action

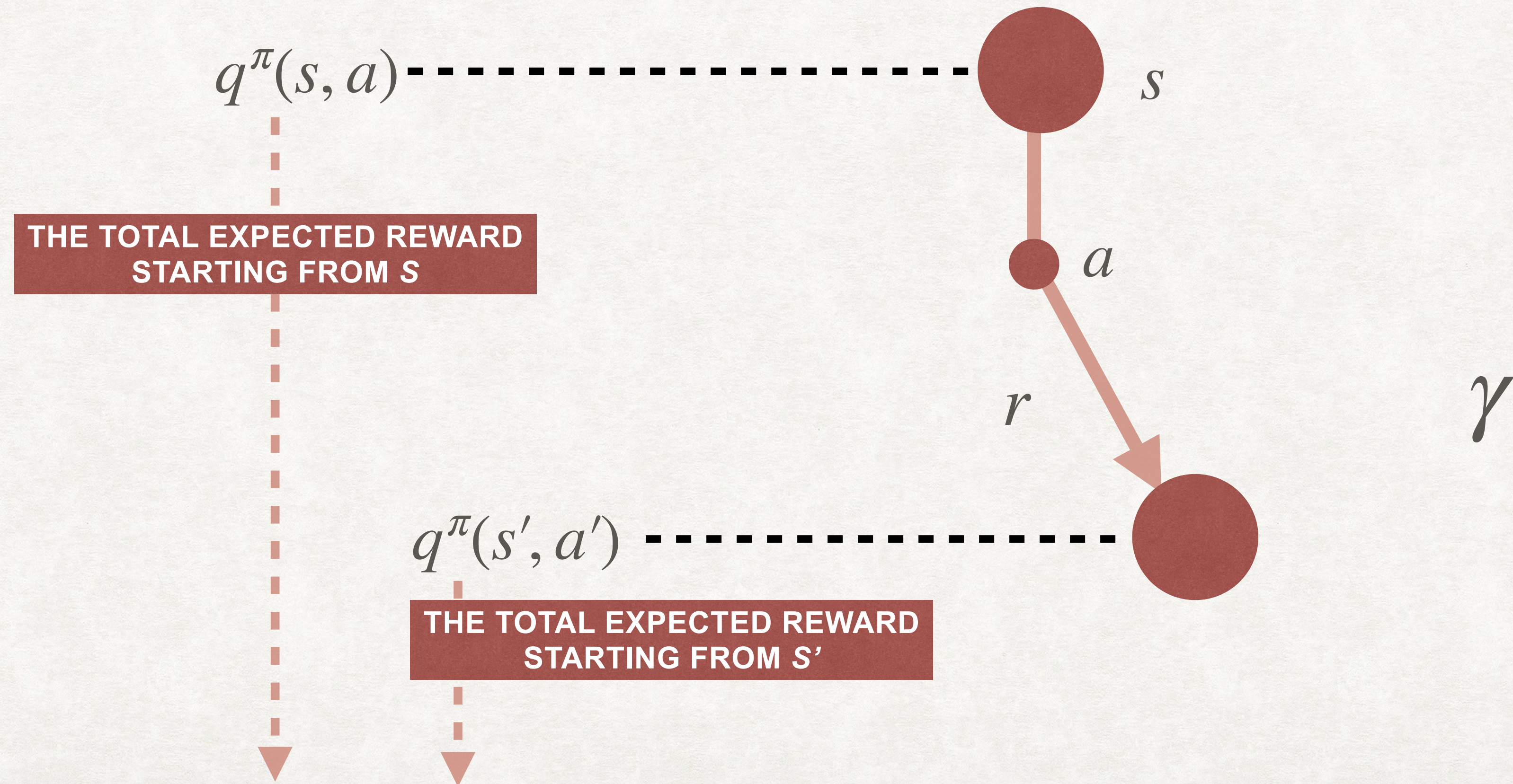
Next state-action

(Stochastic) immediate reward due to action a from state s

FUTURE REWARDS

ACTION-VALUE FUNCTIONS

$$q^{\pi}(s, a) = \mathbb{E}_{\pi}[r + \gamma q^{\pi}(s', a') \mid s, a]$$



FUTURE REWARDS

OPTIMALITY & BELLMAN EQUATION FOR ACTION-VALUES

$$q^*(s, a) = \max_{\pi} q^{\pi}(s, a)$$

One policy better or equal than any other

Greedy policy *

$$q^{\pi}(s, a) = \mathbb{E}_{\pi}[r + \gamma q^{\pi}(s', a') \mid s, a] \quad \pi = *$$

$$q^*(s, a) = \mathbb{E}[r + \gamma \max_{a'} q^*(s', a') \mid s, a]$$

IMMEDIATE REWARDS - A REMINDER

BANDITS



Please draw me a one-arm bandit in the concept of reinforcement learning.

$$\mathbb{E}[R] = \sum_i r_i P(R = r_i)$$

$$q^* = \mathbb{E}[R]$$

$$\mathbb{E}[R - q^*] = 0$$

Approximate with Q

$$E = \frac{1}{2}(r - Q)^2$$

Reward sample

DALL·E

IMMEDIATE REWARDS - A REMINDER

BANDITS



$$E = \frac{1}{2}(r - Q)^2$$

$$\frac{\partial E}{\partial Q} = -(r - Q)$$

$$\Delta Q = -\eta \frac{\partial E}{\partial Q} = \eta(r - Q)$$

Please draw me a one-arm bandit in the concept of reinforcement learning.

DALL·E

FUTURE REWARDS

SARSA

$$q^{\pi}(s, a) = \mathbb{E}_{\pi}[r + \gamma q^{\pi}(s', a') \mid s, a]$$

$$\mathbb{E}_{\pi}[r + \gamma q^{\pi}(s', a') - q^{\pi}(s, a) \mid s, a] = 0$$

$$E = \frac{1}{2} \left(r + \gamma Q(s', a') - Q(s, a) \right)^2$$

FUTURE REWARDS

SARSA

$$E = \frac{1}{2} \left(r + \gamma Q(s', a') - Q(s, a) \right)^2$$

$$\frac{\partial E}{\partial Q(s, a)} = - \left(r + \gamma Q(s', a') - Q(s, a) \right)$$

$$\Delta Q = - \eta \frac{\partial E}{\partial Q(s, a)} = \eta \left(r + \gamma Q(s', a') - Q(s, a) \right)$$

FUTURE REWARDS

SARSA: REWARD - “ANTICIPATED REWARD”

$$\Delta Q = \eta \left(\underbrace{r + \gamma Q^\pi(s', a')}_{\text{What I “actually” get}} - \underbrace{Q^\pi(s, a)}_{\text{Anticipated reward}} \right)$$

FUTURE REWARDS

POLICIES

Greedy $a = \operatorname{argmax}_a Q(s, a)$

Optimistic Greedy: initialise Q-values unrealistically high

Epsilon-Greedy : explore with probability epsilon, greedy otherwise

Softmax: $P(a) = \frac{e^{Q(s,a)/\tau}}{\sum_b e^{Q(s,b)/\tau}}$

FUTURE REWARDS

THE SARSA ALGORITHM

1. Initialise $Q(s, a)$ arbitrarily for all $s \in S$ and $a \in A(s)$.
2. Repeat (for each episode):
 - a. Initialise s .
 - b. Choose an action a from s using a policy derived from Q (e.g., ϵ -greedy).
 - c. Repeat (for each step of episode):
 - i. Take action a , observe reward r and next state s' .
 - ii. Choose a' from s' using policy derived from Q (e.g., ϵ -greedy).
 - iii. $Q(s, a) \leftarrow Q(s, a) + \eta * [r + \gamma * Q(s', a') - Q(s, a)]$.
 - iv. $s \leftarrow s'$; $a \leftarrow a'$.
 - d. until s is terminal.

FUTURE REWARDS

THE SARSA ALGORITHM

In 1994, Gavin Rummery and Mahesan Niranjan published a paper titled “Online Q-Learning using Connectionist Systems,” in which they introduced an algorithm they called at the time “Modified Connectionist Q-Learning.” In 1996, Singh and Sutton dubbed this algorithm Sarsa because of the quintuple of events that the algorithm uses: $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$. People often like knowing where these names come from as you will soon see, RL researchers can get pretty creative with these names.

<https://livebook.manning.com/concept/reinforcement-learning/this-algorithm>

FUTURE REWARDS

THE SARSA ALGORITHM

Right after obtaining his Ph.D. in 1995, Gavin became a programmer and later a lead programmer for the company responsible for the series of the Tomb Raider games. Gavin has had a very successful career as a game developer.

Maheesan, who became Gavin's Ph.D. supervisor after the unexpected death of Gavin's original supervisor, followed a more traditional academic career holding lecturer and professor roles ever since his Ph.D. graduation in 1990.

<https://livebook.manning.com/concept/reinforcement-learning/this-algorithm>

FUTURE REWARDS

TEMPORAL DIFFERENCE LEARNING

$$q^*(s, a) = \mathbb{E}[r + \gamma \max_{a'} q^*(s', a') \mid s, a]$$

$$\mathbb{E}[r + \gamma \max_{a'} q^*(s', a') - q^*(s, a) \mid s, a] = 0$$

$$E = \frac{1}{2} \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)^2$$

FUTURE REWARDS

REWARD - “ANTICIPATED REWARD”

$$\Delta Q = \eta \left(\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{What I “actually” get}} - \underbrace{Q(s, a)}_{\text{Anticipated reward}} \right)$$

What I “actually” get

Anticipated reward

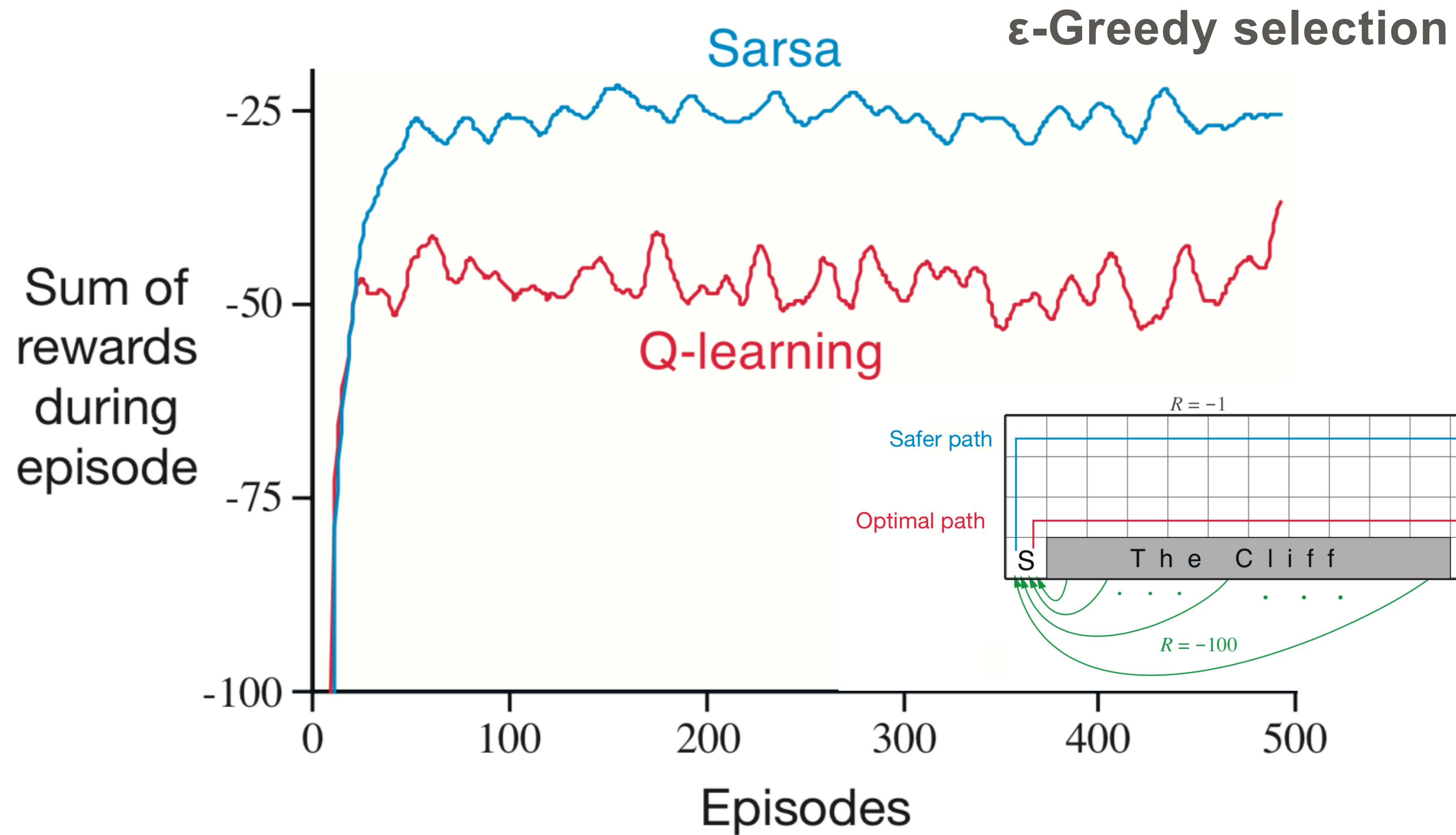
FUTURE REWARDS

THE Q-LEARNING ALGORITHM

1. Initialise $Q(s, a)$ arbitrarily for all $s \in S$ and $a \in A(s)$.
2. Repeat (for each episode):
 - a. Initialise s .
 - b. Repeat (for each step of episode):
 - i. Choose an action a from s using a policy derived from Q (e.g., ϵ -greedy).
 - ii. Take action a , observe reward r and next state s' .
 - iii. $Q(s, a) \leftarrow Q(s, a) + \eta * [r + \gamma * \max_{a'} Q(s', a') - Q(s, a)]$.
 - iv. $s \leftarrow s'$.
 - c. until s is terminal.

FUTURE REWARDS

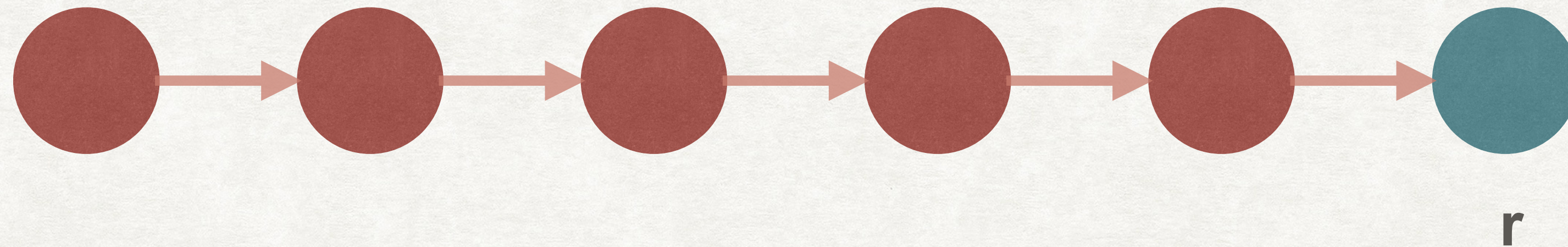
SARSA VS Q-LEARNING



FUTURE REWARDS

SARSA

$$\Delta Q = \eta (r + \gamma Q(s', a') - Q(s, a))$$



A robot walks this corridor and finds reward. How many Q-values will be updated?

FUTURE REWARDS

SARSA: TABULAR ELIGIBILITY TRACE

1. Initialise $Q(s, a)$ arbitrarily for all $s \in S$ and $a \in A(s)$.
2. Initialise eligibility traces $e(s, a)$ for all s, a to zeros.
3. Repeat (for each episode):
 - a. Initialise s .
 - b. Choose an action a from s using policy derived from Q (e.g., ϵ -greedy).
 - c. Repeat (for each step of episode):
 - i. Take action a , observe reward r , and next state s' .
 - ii. Choose a' from s' using policy derived from Q (e.g., ϵ -greedy).
 - iii. For all s, a :
 - Update delta: $\delta \leftarrow r + \gamma * Q(s', a') - Q(s, a)$
 - Update eligibility trace: $e(s, a) \leftarrow e(s, a) + 1$
 - For all s, a :
 - * Update Q : $Q(s, a) \leftarrow Q(s, a) + \eta * \delta * e(s, a)$
 - * Decay eligibility trace: $e(s, a) \leftarrow \gamma\lambda * e(s, a)$
 - iv. $s \leftarrow s'; a \leftarrow a'$.
 - d. until s is terminal.

FUTURE REWARDS

TABULAR ELIGIBILITY TRACE

1. Initialise $Q(s, a)$ arbitrarily for all $s \in S$ and $a \in A(s)$.
2. Initialise **eligibility** traces $e(s, a)$ for all s, a to zeros.
3. Repeat (for each episode):
 - a. Initialise s .
 - b. Repeat (for each step of episode):
 - i. Choose an action a from s using policy derived from Q (e.g., ϵ -greedy).
 - ii. Take action a , observe reward r , and next state s' .
 - iii. Choose a' from s' using policy derived from Q (e.g., ϵ -greedy), but do not take action a' .
 - iv. For all s, a :
 - Update delta: $\delta \leftarrow r + \gamma * \max_{a'} Q(s', a') - Q(s, a)$
 - Update eligibility trace: $e(s, a) \leftarrow \gamma\lambda * e(s, a) + 1$
 - For all s, a :
 - * Update Q : $Q(s, a) \leftarrow Q(s, a) + \eta * \delta * e(s, a)$
 - v. $s \leftarrow s'$.
 - c. until s is terminal.

OPEN PHD POSITION

DR MATT ELLIS

<https://mattoaellis.github.io/>



Matthew Ellis

Lecturer in Machine Learning

[University of Sheffield](#)



Biography

Dr Matthew Ellis is a Lecturer in Machine Learning and member of the Machine Learning Group at the Department of Computer Science at the University of Sheffield. My research interests are in developing energy efficient devices based on spin-electronic devices and reservoir computing. I have previously worked with Prof. Stefano Sanvito at Trinity College Dublin and Prof. Roy Chantrell at the University of York.

Interests

- Neuromorphic Spintronics
- Reservoir Computing
- Artificial Intelligence
- Computational Magnetism

Education

-  PhD in Physics, 2015
University of York
-  MPhys in Theoretical Physics, 2011
University of York

Fully funded PhD Position for National on International Students

FUTURE REWARDS

TEMPORAL DIFFERENCE LEARNING

- **By approximating the action value functions by an estimate Q and by writing an error function based on a form of the Bellman equation we derived two Temporal difference algorithms**
 - **SARSA : State Action Reward State Action - “safe”**
 - **Q-Learning : optimal**

THANK YOU!